



Bài giảng môn học:
Thị giác máy tính (7080518)

CHƯƠNG 2: XỬ LÝ VÀ NÂNG CAO CHẤT LƯỢNG ẢNH (Phần 3)

Đặng Văn Nam
dangvannam@humg.edu.vn

II/ Cải thiện chất lượng ảnh (tiếp)

1. Làm mờ - mịn ảnh

- Làm mờ trung bình
- Làm mờ trung vị
- Làm mờ Gaussian

2. Nhiễu và khử nhiễu trong ảnh

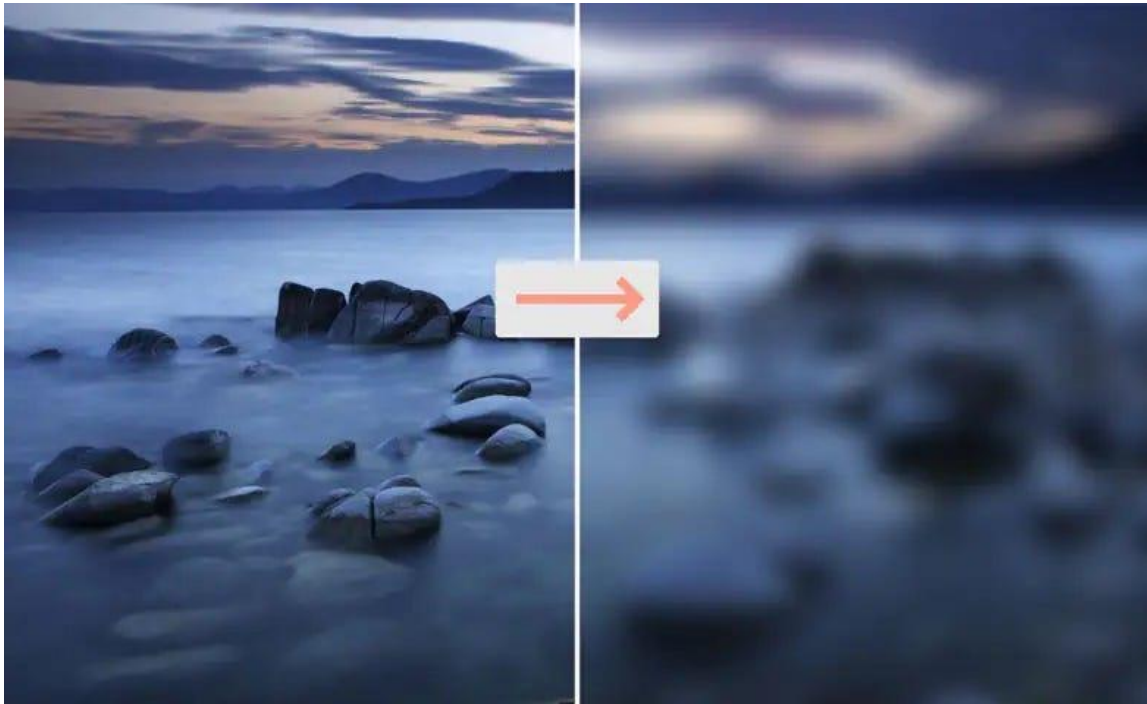
3. Giới thiệu Nhân tích chập

4. Nhân tích chập 2D trong OpenCV

1. Làm mờ – mịn ảnh (blur/smooth images)

Giới thiệu

Làm mờ/mịn ảnh là một kỹ thuật trong xử lý ảnh nhằm **giảm mức độ thay đổi cường độ sáng giữa các điểm ảnh**, từ đó giúp giảm nhiễu (noise) và làm trơn/mịn (smooth) các chi tiết trong ảnh.



Mục tiêu của việc làm mờ ảnh:

- **Giảm nhiễu** xuất hiện do sensor hoặc môi trường
- **Làm trơn bề mặt ảnh**, giảm chi tiết không quan trọng
- **Chuẩn hóa ảnh trước các bước xử lý tiếp theo** (ví dụ: phát hiện biên – edge detection, phân vùng ảnh – segmentation)
- **Tạo hiệu ứng thẩm mỹ** trong chỉnh sửa ảnh

De-noising



Salt and pepper noise



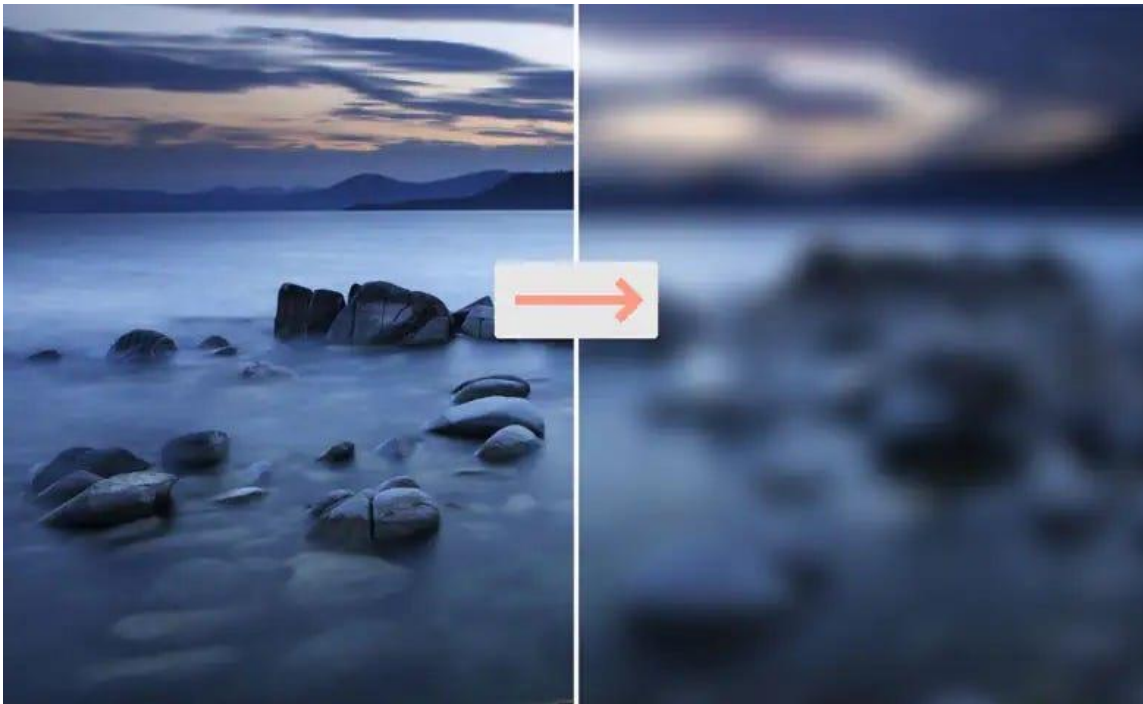
In-painting



Giới thiệu

Nguyên lý chung

Làm mờ ảnh được thực hiện bằng cách **thay thế giá trị pixel trung tâm** bởi **giá trị tổng hợp** (trung bình, trung vị hoặc trọng số) của các pixel trong **vùng lân cận (kernel / window)** xung quanh nó.



Các phương pháp làm mờ/mịn phổ biến:

- Làm mờ trung bình (Mean/Average Blur)
- Làm mờ trung vị (Median Blur)
- Làm mờ Gaussian (Gaussian Blur)
- Làm mờ song phương (Bilateral Filter)

Làm mờ ảnh là bước **tiền xử lý quan trọng**, ảnh hưởng trực tiếp đến chất lượng các thuật toán thị giác máy tính ở các bước phía sau. Việc lựa chọn bộ lọc phụ thuộc vào **đặc điểm nhiều, yêu cầu bảo toàn chi tiết, và mục đích xử lý**.

Làm mờ trung bình

a. Làm mờ trung bình

Làm mờ trung bình (Mean blur): Sử dụng bộ lọc trung bình là một bộ lọc tuyến tính dùng để làm mờ ảnh (smooth) ảnh.

- Giả sử có một cửa sổ lọc kích thước $ksize = (width \times height)$.
- Thực hiện tính trung bình các pixel trong vùng lân cận và gán lại cho pixel trung tâm.

$$ksize = (3 \times 3)$$

$$Value = (23+25+30+25+30+35+45+40+37)/9 = 32.22 = \mathbf{32}$$

Original	23	25	30	35	30	27		Mean Blur
	25	30	35	37	40	55		
	45	40	37	43	45	48		
	38	40	43	42	46	80		
	35	40	42	45	47	95		

a. Làm mờ trung bình

ksize = (3 x 3)

$$\text{Value} = (25+30+35+30+35+37+40+37+43)/9 = 34.67 = \mathbf{35}$$

Original

23	25	30	35	30	27
25	30	35	37	40	55
45	40	37	43	45	48
38	40	43	42	46	80
35	40	42	45	47	95

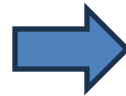
		32	35		

Mean Blur



Original

23	25	30	35	30	27
25	30	35	37	40	55
45	40	37	43	45	48
38	40	43	42	46	80
35	40	42	45	47	95



?	?				
	32	35	37	40	
	37	39	41	48	
	40	41	43	55	



Blur (with a mean filter)

a. Làm mờ trung bình

Phương pháp xử lý trường hợp pixel ở biên hoặc góc của ảnh (border handling)

ksize = (3 x 3)

-	-	-					
-	23	25	30	35	30	27	
-	25	30	35	37	40	55	
	45	40	37	43	45	48	
	38	40	43	42	46	80	
	35	40	42	45	47	95	

?					
	32	35	37	40	
	37	39	41	48	
	40	41	43	55	

0	0	0	0	0	0	0	0
0	23	25	30	35	30	27	0
0	25	30	35	37	40	55	0
0	45	40	37	43	45	48	0
0	38	40	43	42	46	80	0
0	35	40	42	45	47	95	0
0	0	0	0	0	0	0	0

PP1: Zero padding
Các giá trị ngoài biên
gắn bằng giá trị 0

23	23	25	30	35	30	27	27
23	23	25	30	35	30	27	27
25	25	30	35	37	40	55	55
45	45	40	37	43	45	48	48
38	38	40	43	42	46	80	80
35	35	40	42	45	47	95	95
35	35	40	42	45	47	95	95

PP2: Replicate (Sao chép)
Sao chép giá trị biên ra bên ngoài

30	25	30	35	37	40	55	40
25	23	25	30	35	30	27	30
30	25	30	35	37	40	55	40
40	45	40	37	43	45	48	45
40	38	40	43	42	46	80	46
40	35	40	42	45	47	95	47
40	38	40	43	42	46	80	46

PP3: Reflect (Phản chiếu)
Sao chép giá trị biên ra bên ngoài

a. Làm mờ trung bình

Trong OpenCV để làm mờ trung bình sử dụng phương thức:

cv2.blur(src, ksize, borderType)

Trong đó:

- src: ảnh gốc ban đầu
- ksize = (width,height): Kích thước mặt nạ lọc
- borderType: phương pháp xử lý biên
 - cv2.BORDER_CONSTANT (zero padding)
 - cv2.BORDER_REPLICATE (sao chép)
 - cv2.BORDER_REFLECT ~ cv2.BORDER_DEFAULT (phản chiếu – mặc định)

a. Lọc trung bình

```
#Sử dụng bộ lọc trung bình trên ảnh:
ksize = (10,10)

#Thực hiện làm mịn ảnh trung bình:
img_blur = cv2.blur(img_apple,
                    ksize,
                    borderType = cv2.BORDER_REFLECT)

#Hiển thị kết quả:
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(img_apple)
plt.title('Ảnh gốc')

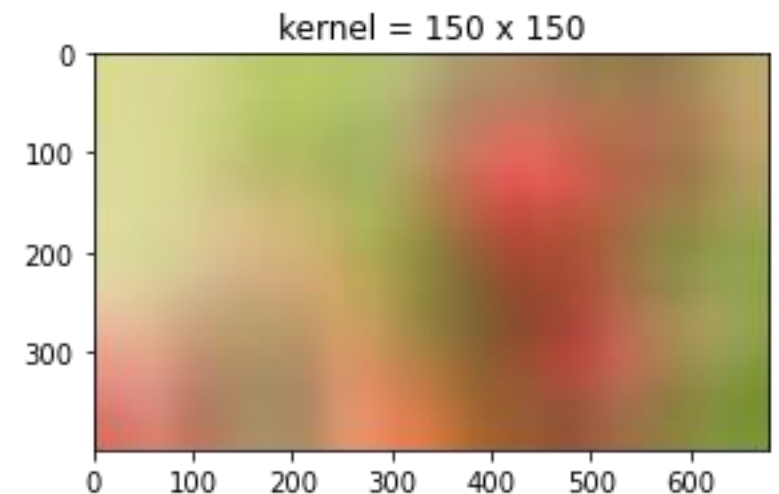
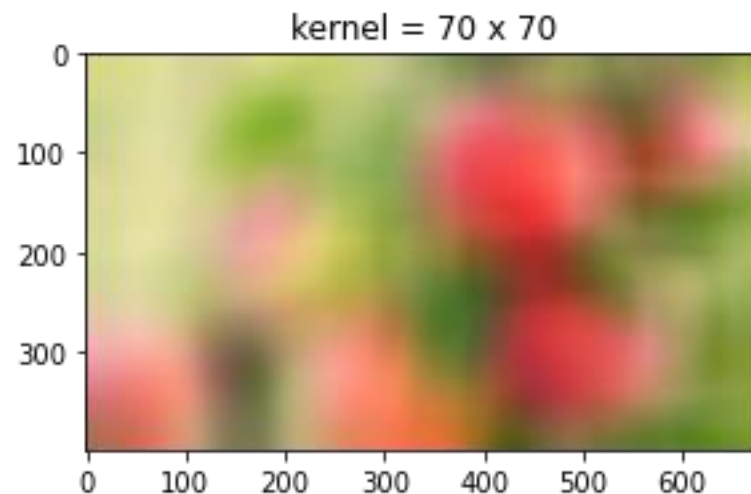
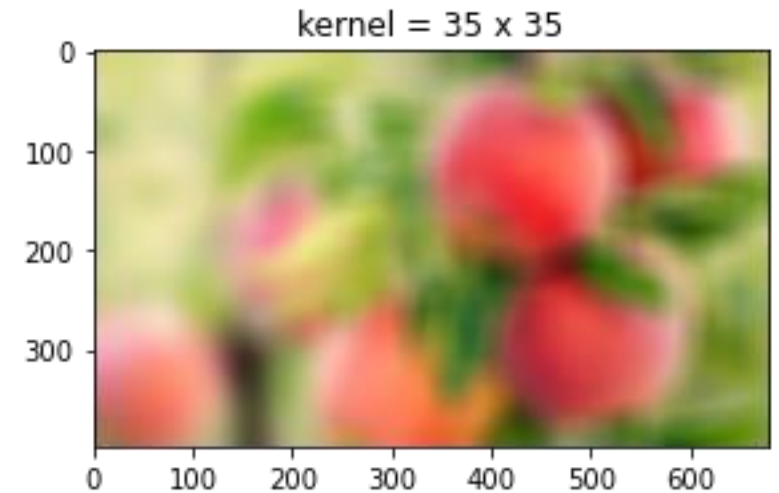
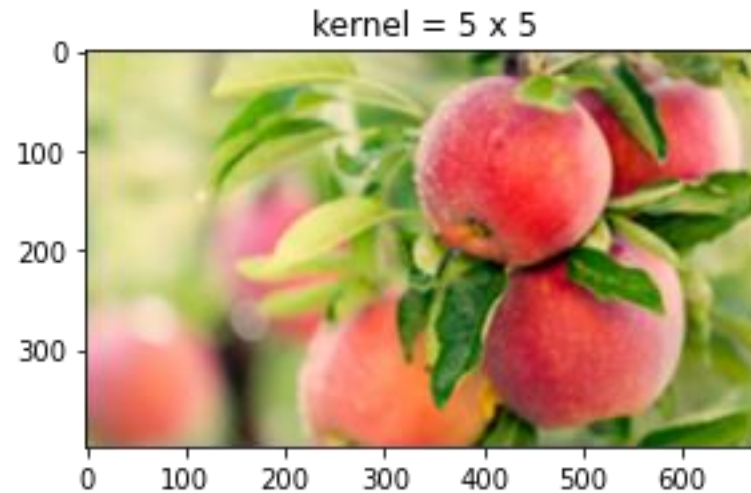
plt.subplot(1,2,2)
plt.imshow(img_blur)
plt.title('Ảnh làm mịn Trung bình (10,10)')
plt.show()
```



Thuộc tính	Mô tả
Loại lọc	Lọc tuyến tính theo trung bình cộng
Tác dụng chính	Giảm nhiễu ngẫu nhiên (noise), làm trơn – làm mịn ảnh
Ảnh hưởng phụ	Làm mất chi tiết và làm mờ biên (edges)
Độ mờ phụ thuộc	Kích thước ksize càng lớn → ảnh càng mờ

a. Làm mờ trung bình

Kích thước mặt nạ lọc càng lớn, ảnh càng mờ:



Làm mờ trung vị

b. Làm mờ trung vị

Làm mờ trung vị (Median blur).

- Là một bộ lọc phi tuyến, dùng để làm trơn ảnh và khử nhiễu bằng cách thay thế giá trị của mỗi pixel bằng giá trị trung vị (median) của các điểm ảnh trong một cửa sổ lân cận
- Cách thức thực hiện:
 - Lấy tất cả các pixel trong cửa sổ lân cận
 - Sắp xếp chúng theo thứ tự tăng dần
 - Lấy giá trị trung vị (phần tử ở giữa danh sách) thay cho pixel trung tâm

1 2 4 7 9



Sample median

b. Làm mờ trung vị

ksize = (3 x 3)

Sắp xếp: 105, 106, 107, 109, **110**, 110, 110, 111, 112 = **110**

Original

110	112	109	109	135	140
105	111	110	110	128	137
107	106	110	116	134	48
102	109	114	121	132	143
100	90	88	100	105	108

	110				

Median Blur



 Median Formula = $\frac{(n + 1)}{2}$



b. Làm mờ trung vị

$ksize = (3 \times 3)$

Sắp xếp: 106, 109, 109, 110, **110**, 110, 111, 112, 116 = **110**

Original

110	112	109	109	135	140
105	111	110	110	128	137
107	106	110	116	134	48
102	109	114	121	132	143
100	90	88	100	105	108

		110	110		

Median Blur

b. Làm mờ trung vị

Trong OpenCV để làm mờ trung vị sử dụng phương thức:

cv2.medianBlur(src, ksize)

Trong đó:

- src: ảnh gốc ban đầu
- ksize: Kích thước mặt nạ lọc, một giá trị phải là một số lẻ lớn hơn 1 (vd: 3, 5, 7...)

Lưu ý: Không có thuộc tính `borderType` – Cố định theo cơ chế phản chiếu gương (Reflect) bằng cách lấy các pixel ở gần biên và phản chiếu đối xứng ra bên ngoài.

b. Làm mờ trung vị

#Sử dụng bộ lọc trung vị trên ảnh:

`ksize = 11`

#Thực hiện làm mịn trung vị:

```
img_median = cv2.medianBlur(img_apple,
                             ksize)
```

#hiển thị kết quả:

```
plt.figure(figsize=(10,5))
```

```
plt.subplot(1,2,1)
```

```
plt.imshow(img_apple)
```

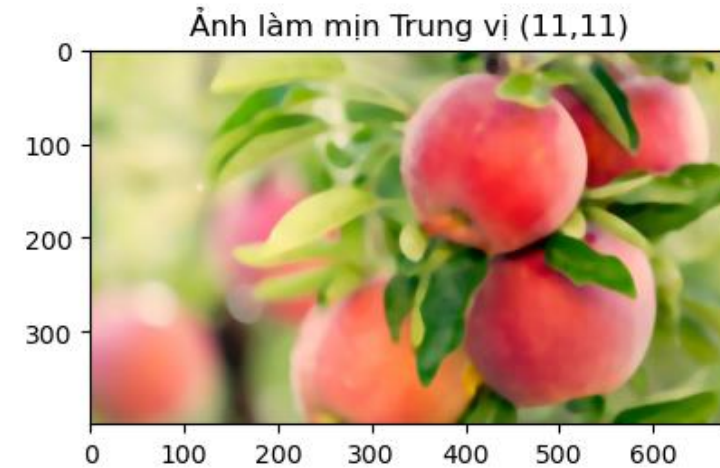
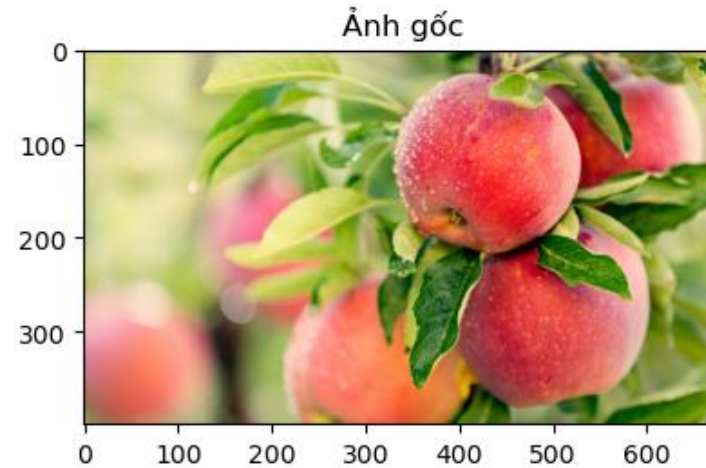
```
plt.title('Ảnh gốc')
```

```
plt.subplot(1,2,2)
```

```
plt.imshow(img_median)
```

```
plt.title('Ảnh làm mịn Trung vị (11,11)')
```

```
plt.show()
```



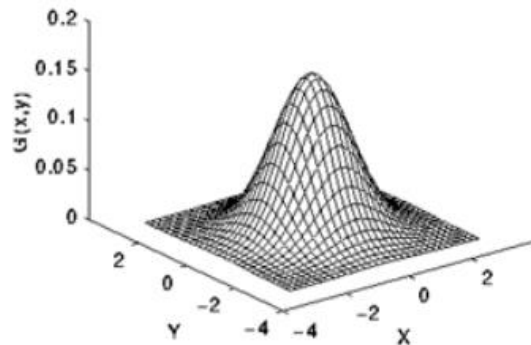
Thuộc tính	Mô tả
Loại lọc	Phi tuyến (non-linear)
Tác dụng mạnh nhất	Khử nhiễu muối tiêu (salt-and-pepper noise)
Giữ biên tốt hơn	So với lọc trung bình và Gaussian
Kích thước kernel	Luôn là số lẻ (3, 5, 7, ...)

Làm mờ Gaussian

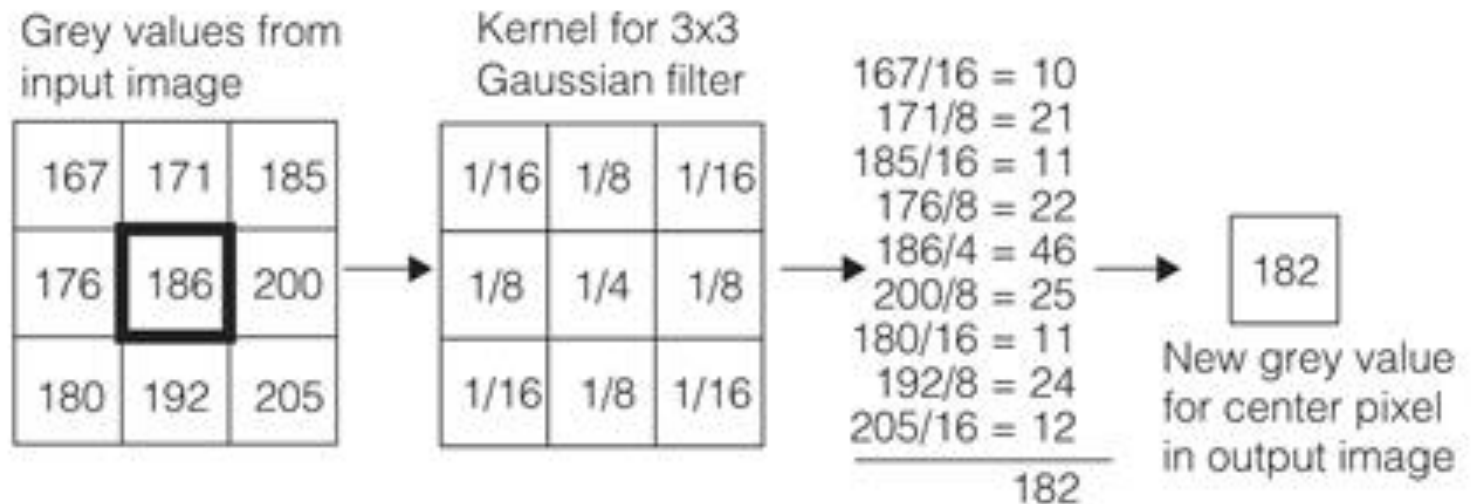
c. Làm mờ Gaussian

Làm mờ Gaussian: Sử dụng Bộ lọc Gaussian (Gaussian Blur) là một bộ lọc làm trơn ảnh tuyến tính có trọng số, trong đó mỗi điểm ảnh đầu ra được tính bằng trung bình có trọng số của các điểm ảnh lân cận, với trọng số được xác định bởi hàm phân phối chuẩn (Gaussian distribution).

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$



Pixel trong vùng lân cận được xác định bằng **hàm Gaussian 2D**:



c. Lọc Gaussian

Trong OpenCV để lọc Gaussian sử dụng phương thức:

`cv2.gaussianBlur(src, ksize, sigmaX, sigmaY, borderType)`

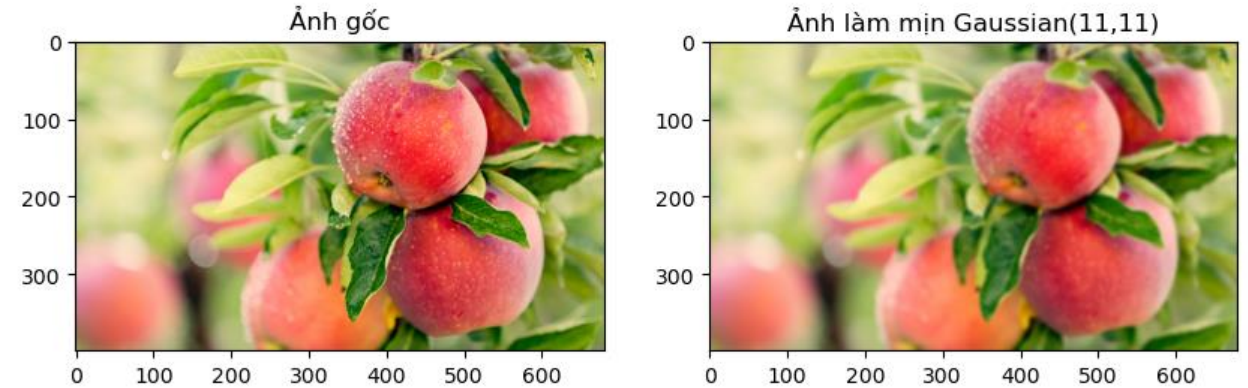
Trong đó:

- src: ảnh gốc ban đầu
- ksize: Kích thước mặt nạ lọc, phải là số dương lẻ.
- sigmaX: Độ lệch chuẩn theo trục X (Điều chỉnh độ mờ chính)
- sigmaY: Độ lệch chuẩn theo trục Y (thường để mặc định = sigmaX)
- borderType: Phương pháp xử lý biên

Lưu ý: Nếu $\text{sigmaX} = 0$, OpenCV tự chọn giá trị tối ưu dựa trên ksize.)

c. Lọc Gaussian

```
#Sử dụng bộ lọc Gaussian trên ảnh:  
ksize = (11,11)  
  
#Thực hiện làm mịn ảnh Gaussian:  
img_gaus = cv2.GaussianBlur(img_apple,  
                             (11,11),  
                             0,  
                             0,  
                             borderType=cv2.BORDER_REPLICATE)  
  
#Hiển thị kết quả:  
plt.figure(figsize=(10,5))  
plt.subplot(1,2,1)  
plt.imshow(img_apple)  
plt.title('Ảnh gốc')  
  
plt.subplot(1,2,2)  
plt.imshow(img_gaus)  
plt.title('Ảnh làm mịn Gaussian(11,11)')  
plt.show()
```



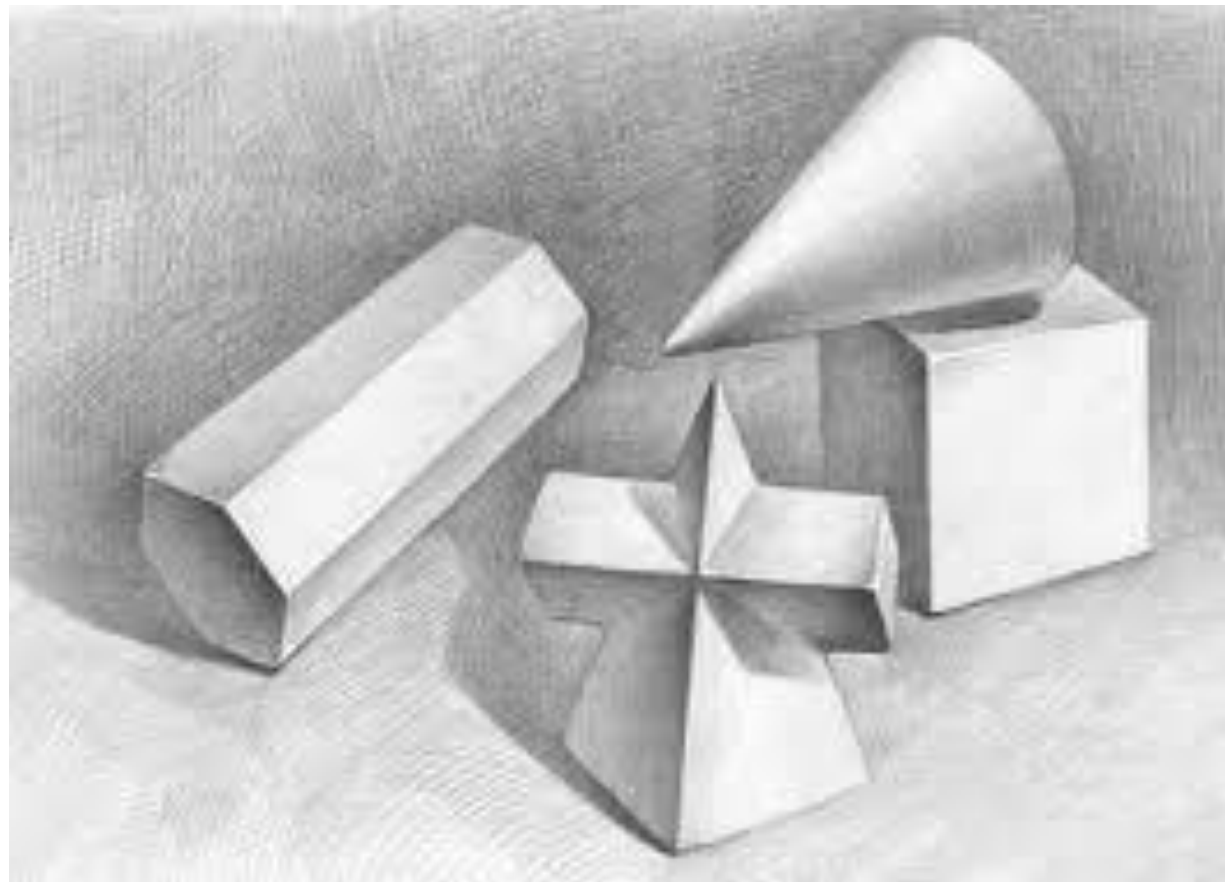
So sánh 3 bộ lọc làm mờ ảnh:

Phương pháp	Cơ chế	Ưu điểm	Nhược điểm	Trường hợp sử dụng
Trung bình (blur)	Trung bình cộng đều	Nhanh, đơn giản	Làm mất chi tiết, mờ cạnh mạnh	Làm trơn tổng quát
Trung vị (medianBlur)	Lấy trung vị	Khử nhiễu muối tiêu rất tốt, giữ biên tốt	Chậm hơn, không hiệu quả cho nhiễu Gaussian	Ảnh chứa nhiễu salt-and-pepper
Gaussian (GaussianBlur)	Trung bình có trọng số Gaussian	Mờ mềm, tự nhiên, giữ biên tốt hơn	Chậm hơn blur	Giảm nhiễu nhẹ, xử lý trước khi phát hiện biên

Thực hành số 2.8

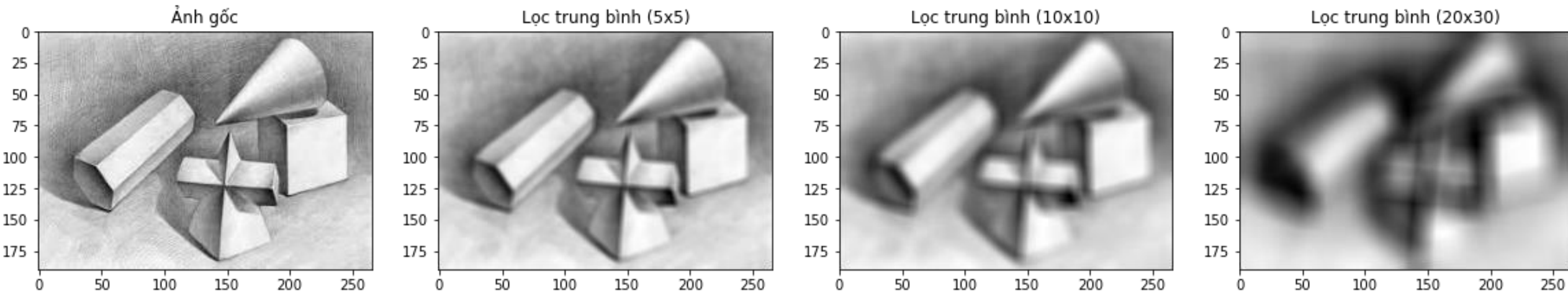
Thực hành 2.8

Yêu cầu 1: Sinh viên đọc ảnh images/Thuchanh2_8.jpeg và hiển thị ảnh



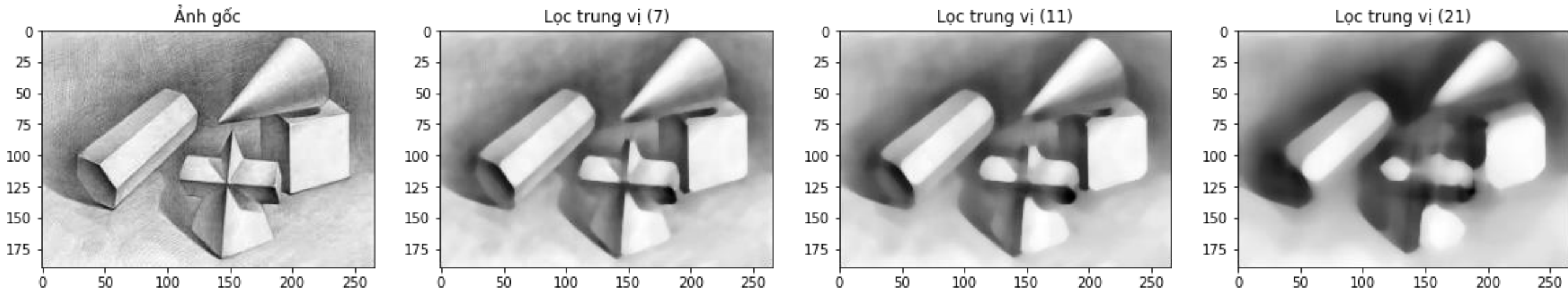
Thực hành 2.8

Yêu cầu 2: Sử dụng phương pháp làm mịn trung bình với kernel 5x5, 10x10, 20x30 và hiển thị kết quả



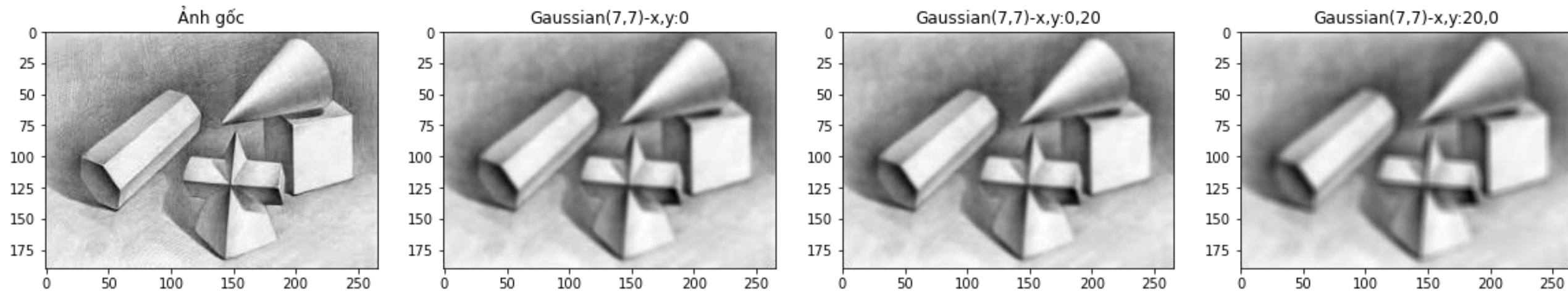
Thực hành 2.8

Yêu cầu 3: Sử dụng phương pháp làm mịn trung vị với $k = 7, 11, 21$ và hiển thị kết quả



Thực hành 2.8

Yêu cầu 4: Sử dụng phương pháp làm mịn Gaussian với kernel = (7,7) và sigmaX, sigmaY lần lượt là (0,0); (0,20); (20,0) và hiển thị kết quả



2. Nhiều và khử nhiễu trong ảnh

Nhiều trong ảnh là gì?

Nhiều (Noise) là các tín hiệu không mong muốn xuất hiện trong ảnh, làm biến đổi giá trị cường độ sáng hoặc màu sắc của các điểm ảnh, khiến ảnh bị mờ, kém rõ nét hoặc mất chi tiết quan trọng.

Original Image (Left) and Noisy Image (Right)



Nhiều thường phát sinh trong quá trình:

- Cảm biến máy ảnh thu nhận tín hiệu (sensor noise)
- Truyền dẫn dữ liệu (channel noise)
- Ánh sáng môi trường (low-light conditions)
- Xử lý và nén ảnh (compression artifacts)

Nhiều trong ảnh là gì?

Ảnh nhiễu được biểu diễn bằng biểu thức:

$$g(x,y) = f(x,y) + \alpha(x,y)$$

Trong đó:

- $f(x,y)$: Ảnh gốc
- $\alpha(x,y)$: Nhiễu
- $g(x,y)$: Ảnh sau khi bị nhiễu tác động.



Original Image (Left) and Noisy Image (Right)



Một số loại nhiễu trong ảnh

Loại nhiễu	Đặc điểm	Nguyên nhân	Ví dụ
Nhiều Gaussian	Xuất hiện ngẫu nhiên, phân bố theo dạng chuông	Cảm biến điện tử, môi trường ánh sáng yếu	Nhiều mờ hạt mịn



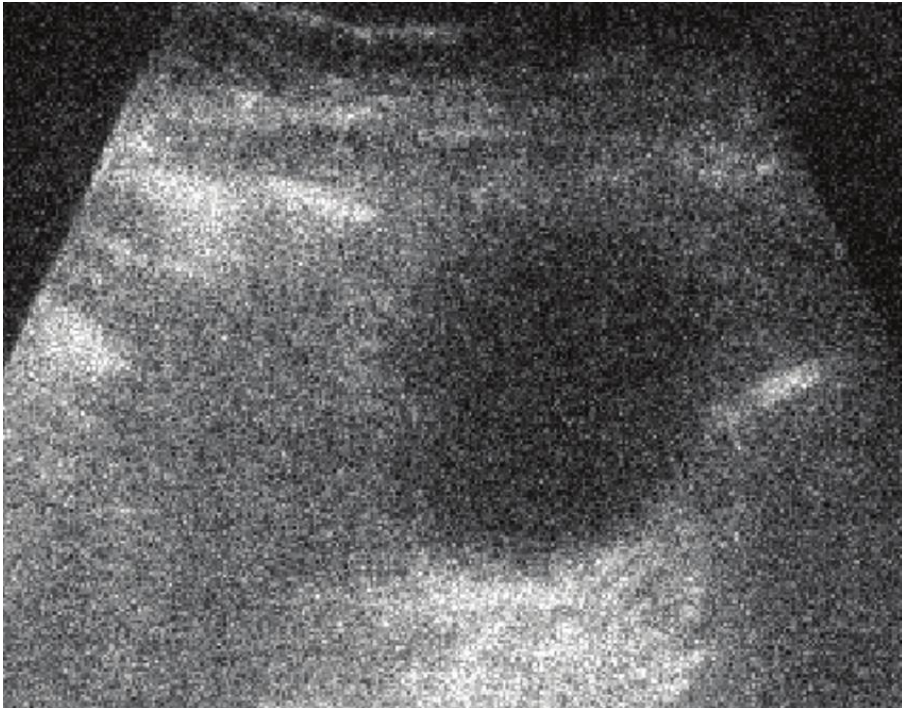
Một số loại nhiễu trong ảnh

Loại nhiễu	Đặc điểm	Nguyên nhân	Ví dụ
Nhiều Muối – Tiêu (Salt & Pepper)	Xuất hiện các điểm đen - trắng rõ rệt	Truyền tín hiệu lỗi, hỏng pixel	Xuất hiện đốm trắng/đen rải rác



Một số loại nhiễu trong ảnh

Loại nhiễu	Đặc điểm	Nguyên nhân	Ví dụ
Nhiều Speckle	Gặp trong ảnh radar, siêu âm; dạng nhiễu nhân tử	Giao thoa sóng	Ảnh y tế, ảnh SAR bị lốm đốm



Khử nhiễu trong ảnh (khôi phục ảnh)

Khử nhiễu là quá trình **làm giảm hoặc loại bỏ nhiễu** khỏi ảnh, trong khi **bảo toàn tối đa chi tiết và cấu trúc quan trọng** (đặc biệt là **các biên và đường nét**).

Mục đích: là để thu nhận được ảnh càng gần với ảnh gốc càng tốt

Một số phương pháp loại bỏ nhiễu:

- ❖ Bộ Lọc làm mờ: Trung bình số học, hình học, Harmonic, Contra harmonic
- ❖ Bộ Lọc thứ tự thống kê: Trung vị, Max, Min, trung bình cắt alpha.
- ❖ Bộ Lọc thích nghi

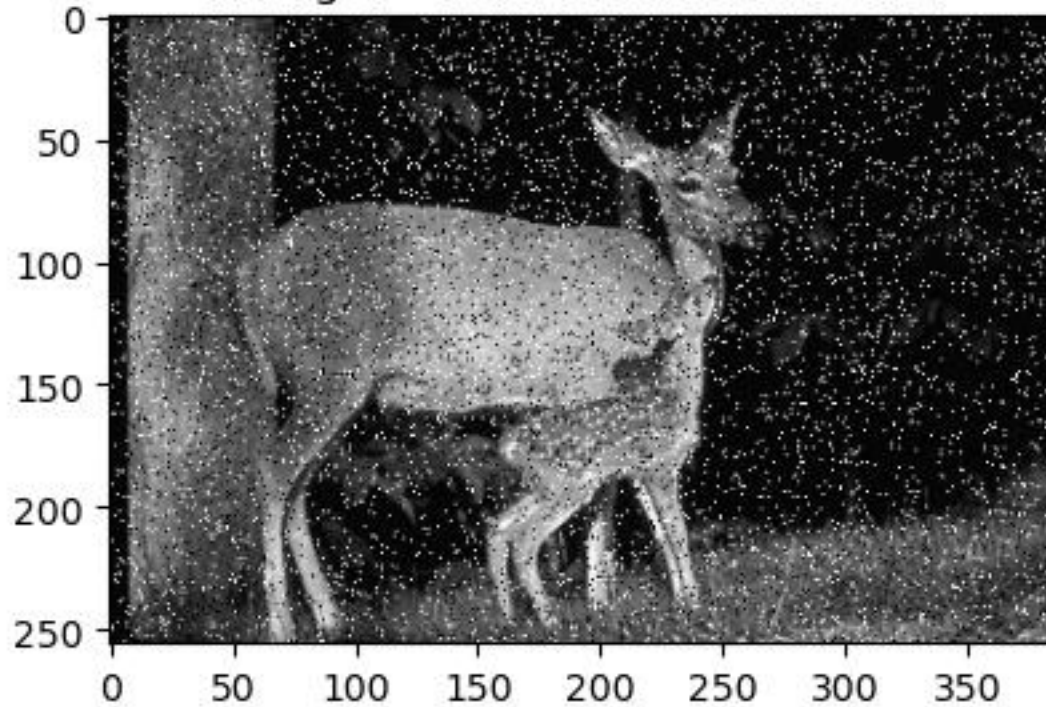
Không có một bộ lọc nào phù hợp cho mọi trường hợp.

Việc lựa chọn kỹ thuật khử nhiễu phụ thuộc vào: loại nhiễu, mức độ nhiễu và mục tiêu phân tích ảnh.

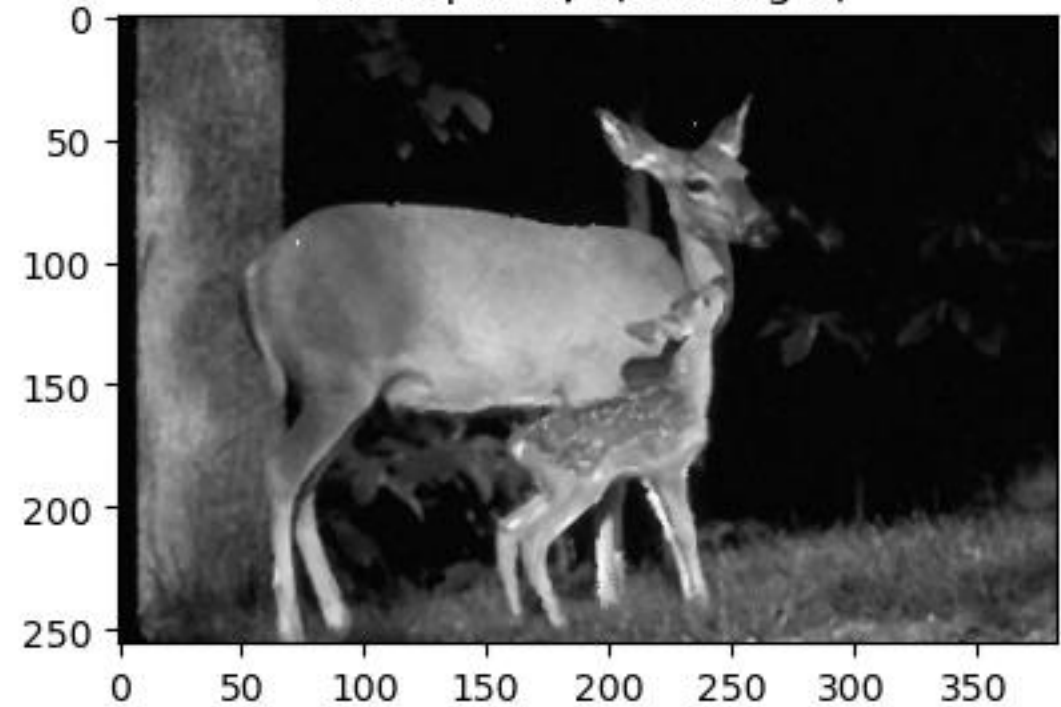


Ví dụ Khử nhiễu trong ảnh (khôi phục ảnh)

Ảnh gốc - Nhiễm nhiễu muối tiêu



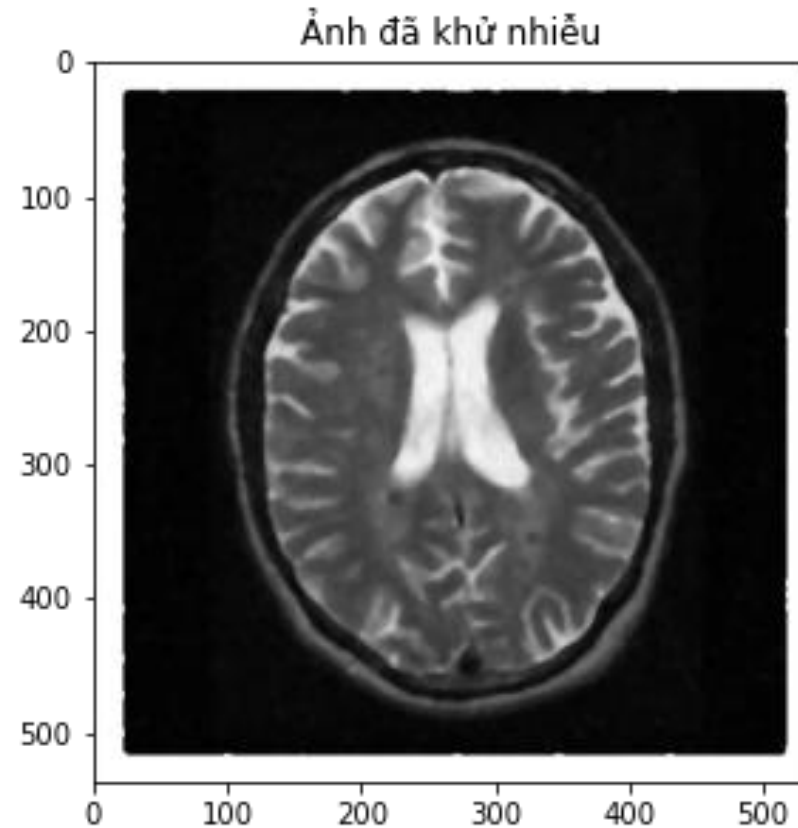
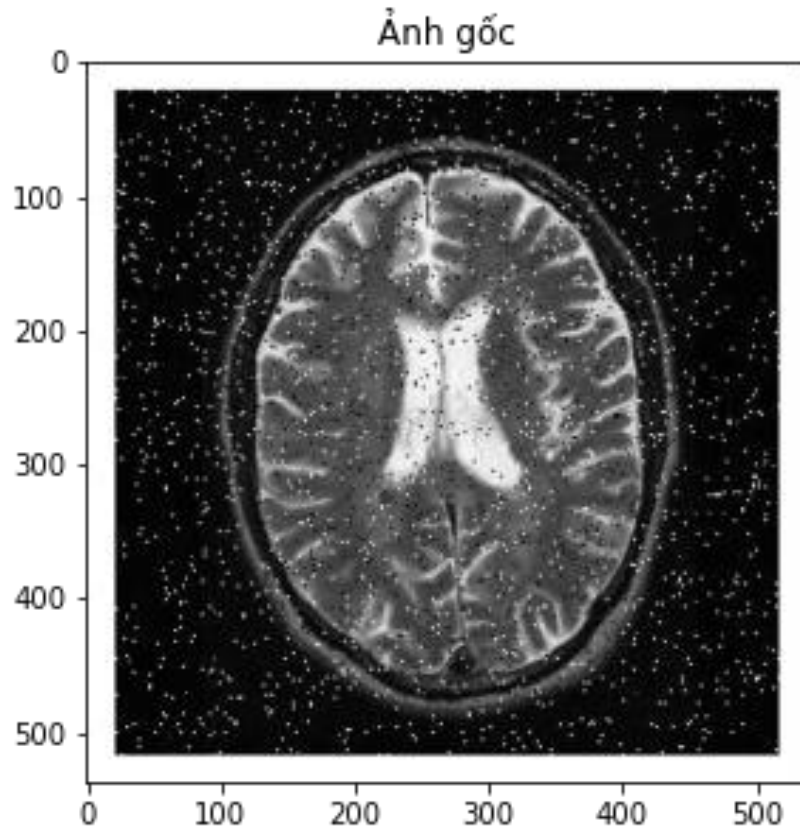
Ảnh qua bộ lọc trung vị



Thực hành số 2.9

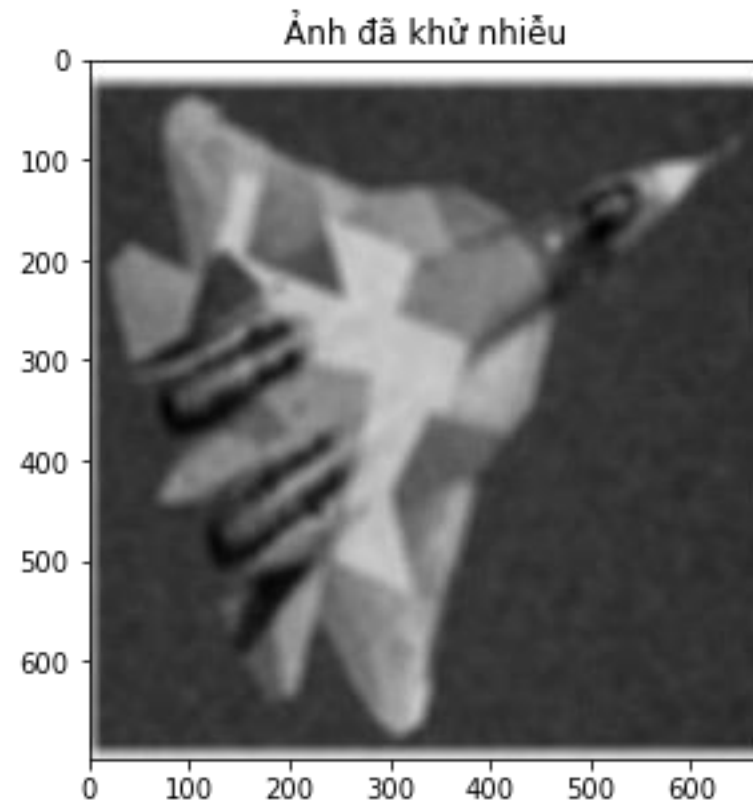
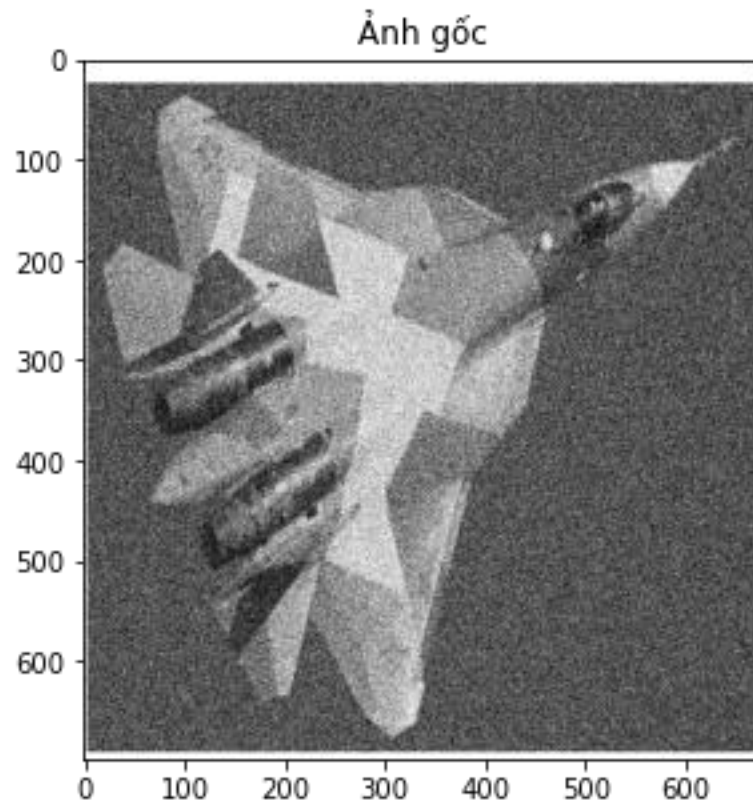
Thực hành 2.9

Yêu cầu 1: Sinh viên đọc ảnh images/Thuchanh2_9_1.jpeg. Lựa chọn phương pháp khử nhiễu và tham số phù hợp để thu được ảnh khử nhiễu tốt nhất. Lưu ảnh đã xử lý vào thư mục images/Saves/MSV_brain.png



Thực hành 2.9

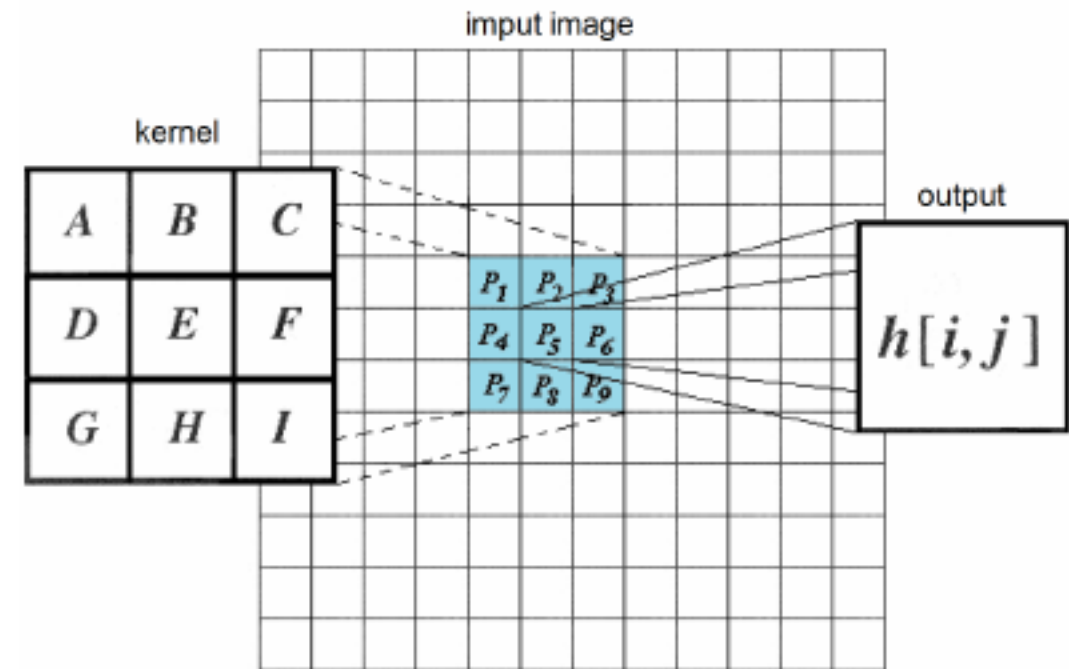
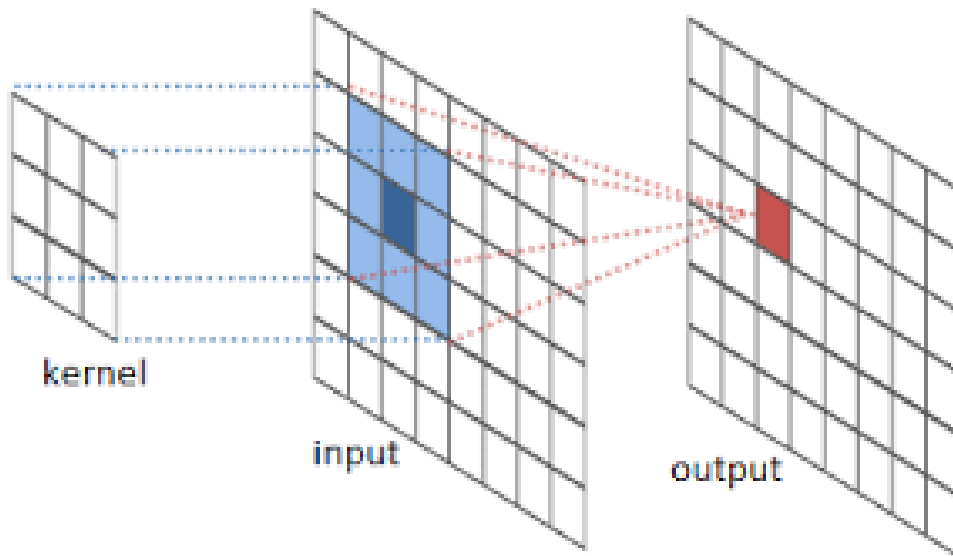
Yêu cầu 2: Sinh viên đọc ảnh images/Thuchanh2_9_2.png. Lựa chọn phương pháp khử nhiễu và các tham số phù hợp để thu được ảnh khử nhiễu tốt nhất. Lưu ảnh đã xử lý vào thư mục images/Saves/MSV_plane.png



3. Giới thiệu Nhân tích chập

Mặt nạ lọc:

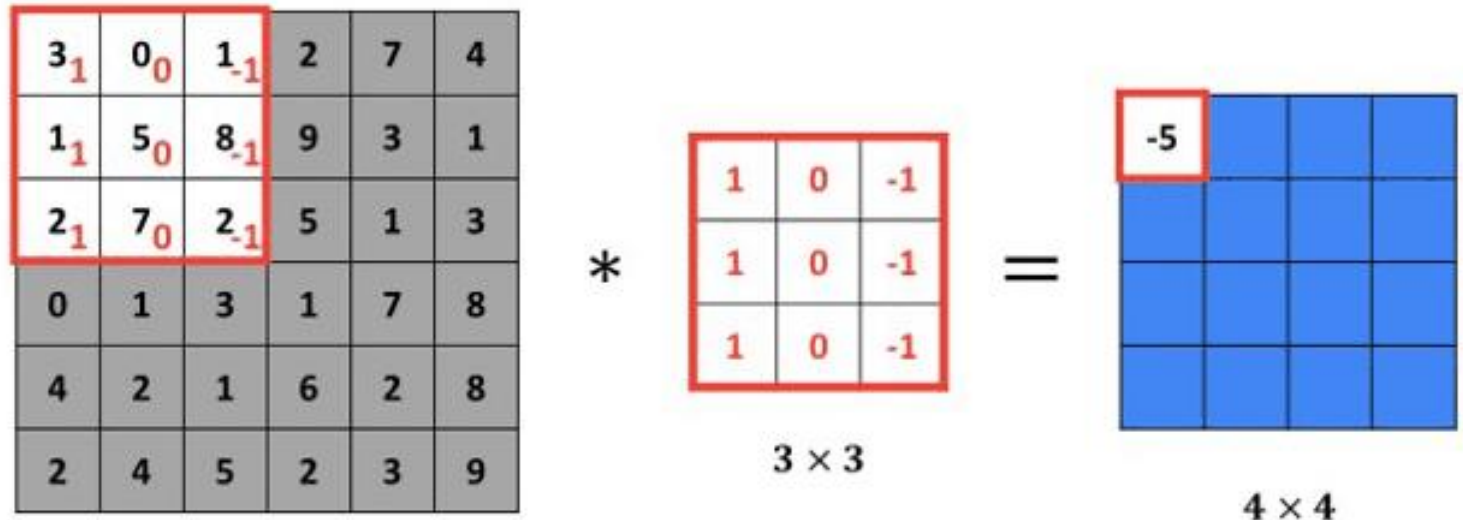
- Mặt nạ lọc còn được gọi là kernel, filter, mask, là một ma trận vuông có kích thước 3x3, 4x4, 5x5, 7x7...
- Mặt nạ lọc di chuyển trên ảnh và thao tác lân cận với điểm ảnh.



$$h(i,j)=A*P_1+B*P_2+C*P_3+D*P_4+E*P_5+F*P_6+G*P_7+H*P_8+I*P_9$$

Nhân tích chập

- Nhân tích chập là cho ma trận ảnh nhân với một ma trận lọc (Kernel). Ma trận lọc (Kernel) còn có thể được gọi là cửa sổ chập (trong phép nhân chập), cửa sổ lọc, mặt nạ,...Việc nhân ảnh với ma trận lọc giống như việc trượt ma trận lọc theo hàng trên ảnh và nhân với từng vùng của ảnh, cộng các kết quả lại tạo thành kết quả của điểm ảnh trung tâm.



$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

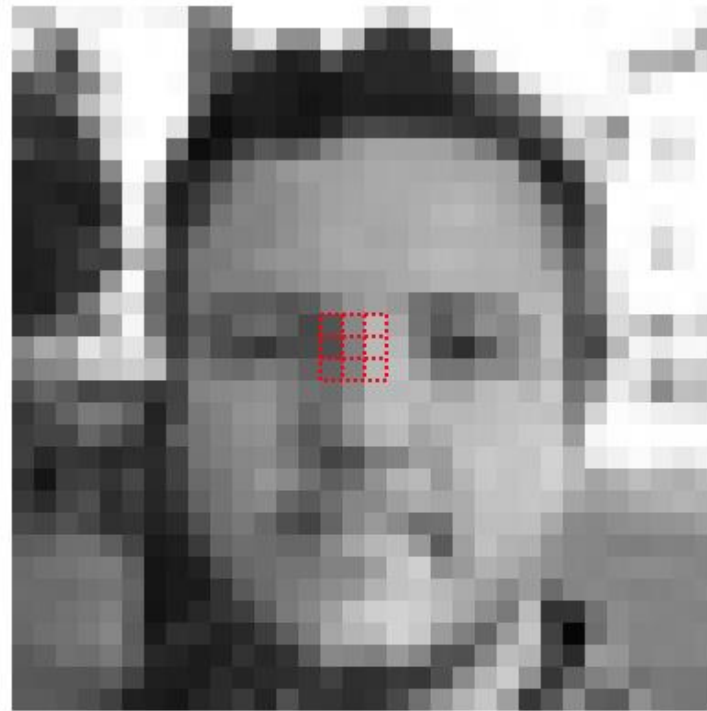
Nhân tích chập

- Mặt nạ lọc

sharpen

0	-1	0
-1	5	-1
0	-1	0

Nhân tích chập



input image

$$\begin{pmatrix}
 \begin{matrix} 94 \\ \times 0 \end{matrix} + \begin{matrix} 147 \\ \times -1 \end{matrix} + \begin{matrix} 191 \\ \times 0 \end{matrix} \\
 + \begin{matrix} 88 \\ \times -1 \end{matrix} + \begin{matrix} 139 \\ \times 5 \end{matrix} + \begin{matrix} 192 \\ \times -1 \end{matrix} \\
 + \begin{matrix} 110 \\ \times 0 \end{matrix} + \begin{matrix} 139 \\ \times -1 \end{matrix} + \begin{matrix} 191 \\ \times 0 \end{matrix}
 \end{pmatrix}
 = 129$$

kernel: sharpen

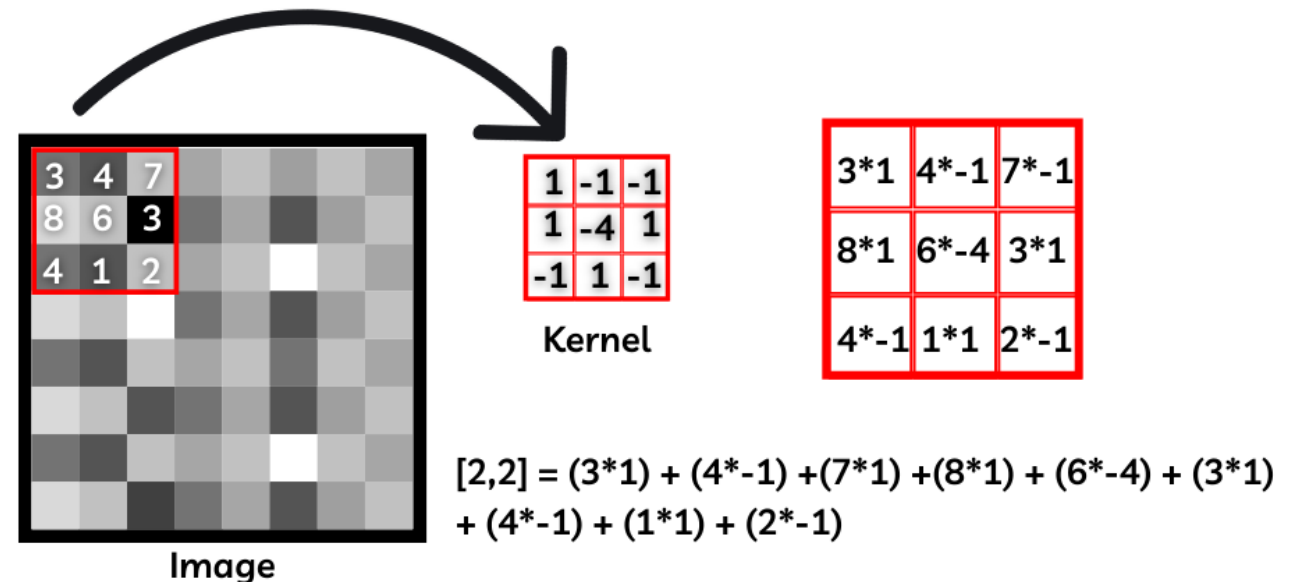


output image

4. Nhận tích chập 2D với OpenCV

Nhân tích chập 2D với OpenCV

- Nguyên tắc chung của các phương pháp lọc là cho ma trận ảnh nhân với một ma trận lọc (Kernel). Ma trận lọc lọc (Kernel) còn có thể được gọi là cửa sổ chập (trong phép nhân chập), cửa sổ lọc, mặt nạ,...Việc nhân ảnh với ma trận lọc giống như việc trượt ma trận lọc theo hàng trên ảnh và nhân với từng vùng của ảnh, cộng các kết quả lại tạo thành kết quả của điểm ảnh trung tâm.
- Với mỗi phép lọc ta có những ma trận lọc (Kernel) khác nhau, không có một quy định cụ thể nào cho việc xác định M. Kích thước ma trận M là một số lẻ. Ví dụ: 3x3, 5x5.



Nhân tích chập 2D với OpenCV

OpenCV hỗ trợ phương thức lọc ảnh 2D:

`cv2.filter2D(src, ddepth, kernel, anchor, delta, borderType)`

Trong đó:

- `src`: ảnh gốc muốn áp dụng bộ lọc
- `ddepth`: Độ sâu của ảnh đầu ra; -1: độ sâu của ảnh đầu ra bằng độ sâu của ảnh đầu vào.
- `kernel`: ma trận lọc
- `anchor`: Vị trí tâm của kernel (mặc định là tâm của ma trận)
- `delta`: Giá trị cộng thêm vào mỗi pixel sau khi lọc (mặc định = 0)
- `borderType`: Cách xử lý biên ảnh

Input depth (src.depth())	Output depth (ddepth)
CV_8U	-1/CV_16S/CV_32F/CV_64F
CV_16U/CV_16S	-1/CV_32F/CV_64F
CV_32F	-1/CV_32F
CV_64F	-1/CV_64F

Nhân tích chập 2D với OpenCV

Một số bộ lọc thông dụng:



$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Identity kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Edge detection

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sharpen kernel

-2	-1	0
-1	1	1
0	1	2

Emboss kernel

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Box blur

$$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

Gaussian blurr kernel

Một số bộ lọc thông dụng:

Bộ lọc Identity:

```
1 #Khởi tạo kernel:
2 kernel_identity = np.array([[0,0,0],
3                             [0,1,0],
4                             [0,0,0]])
5 kernel_identity
```

```
array([[0, 0, 0],
       [0, 1, 0],
       [0, 0, 0]])
```

```
1 #Thực hiện lọc ảnh với kernel ở trên:
2 img_identity = cv2.filter2D(img, -1, kernel_identity)
3
4 #Hiển thị ảnh đã xử lý qua bộ lọc:
5 plt.figure(figsize=(10,8))
6 plt.subplot(1,2,1)
7 plt.imshow(img,cmap='gray')
8 plt.title('Ảnh gốc ban đầu')
9
10
11 plt.subplot(1,2,2)
12 plt.imshow(img_identity,cmap='gray')
13 plt.title('Ảnh qua bộ lọc Identity')
14 plt.show()
```

Ảnh gốc



Ảnh qua bộ lọc identity

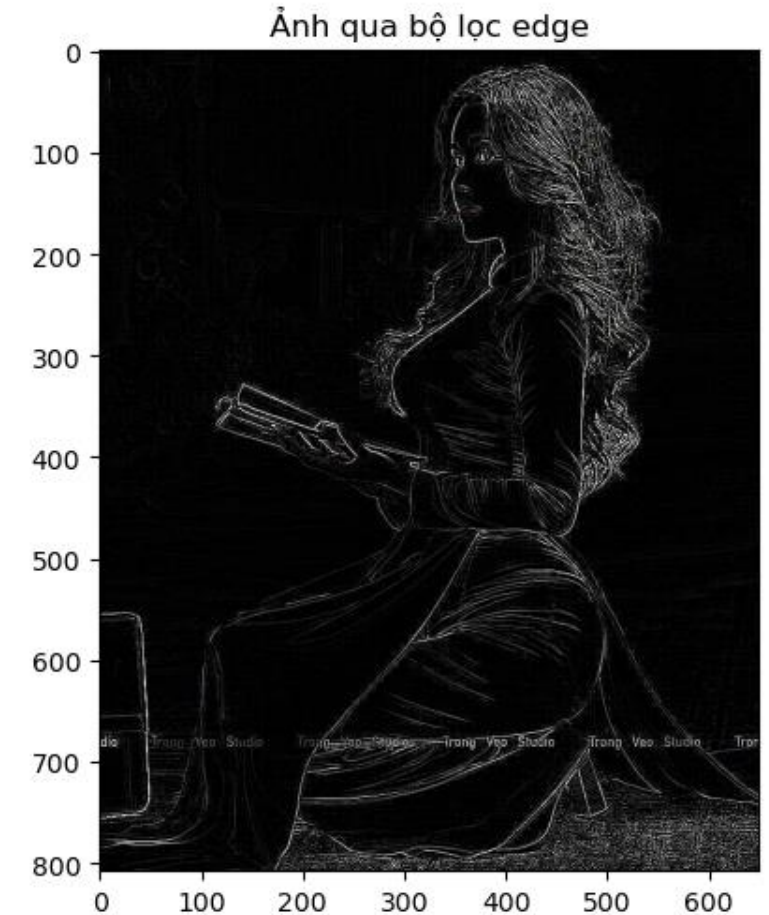


Một số bộ lọc thông dụng:

Bộ lọc Edge:

```
1 #Khởi tạo kernel:
2 kernel_edge = np.array([[ -1, -1, -1],
3                          [ -1,  8, -1],
4                          [ -1, -1, -1]])
5 kernel_edge

array([[ -1, -1, -1],
       [ -1,  8, -1],
       [ -1, -1, -1]])
```



Một số bộ lọc thông dụng:

Bộ lọc sharpen:

```
1 #Khởi tạo kernel:
2 kernel_sharpen = np.array([[ 0, -1, 0],
3                             [-1, 5, -1],
4                             [ 0, -1, 0]])
5 kernel_sharpen

array([[ 0, -1,  0],
       [-1,  5, -1],
       [ 0, -1,  0]])
```



Một số bộ lọc thông dụng:

Bộ lọc emboss:

```
#Khởi tạo kernel:
kernel_emboss = np.array([[ -2, -1, 0],
                          [-1, 1, 1],
                          [ 0, 1, 2]])
print('Mặt nạ lọc emboss: \n', kernel_emboss)
```

Mặt nạ lọc emboss:

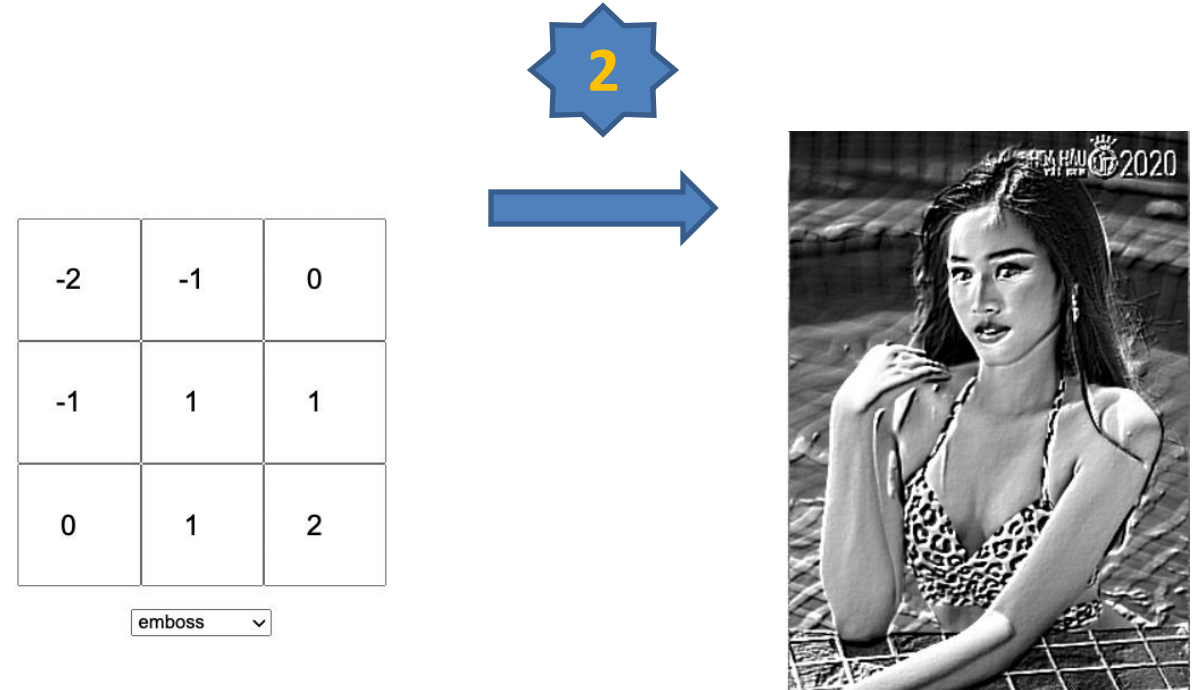
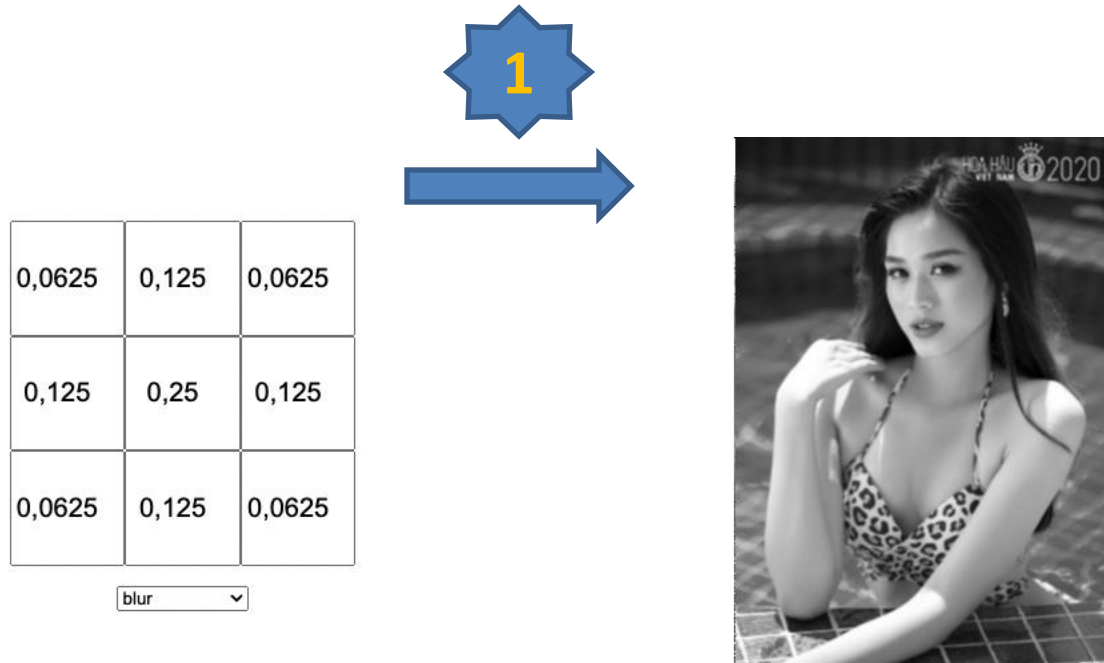
```
[[-2 -1  0]
 [-1  1  1]
 [ 0  1  2]]
```



Thực hành số 2.10

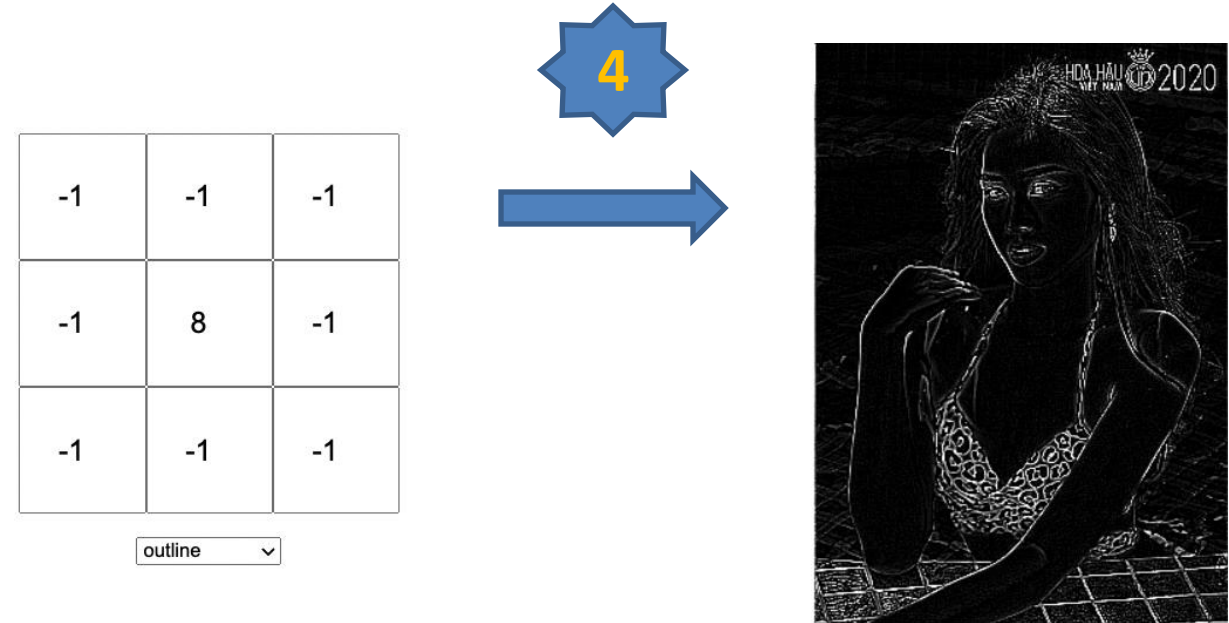
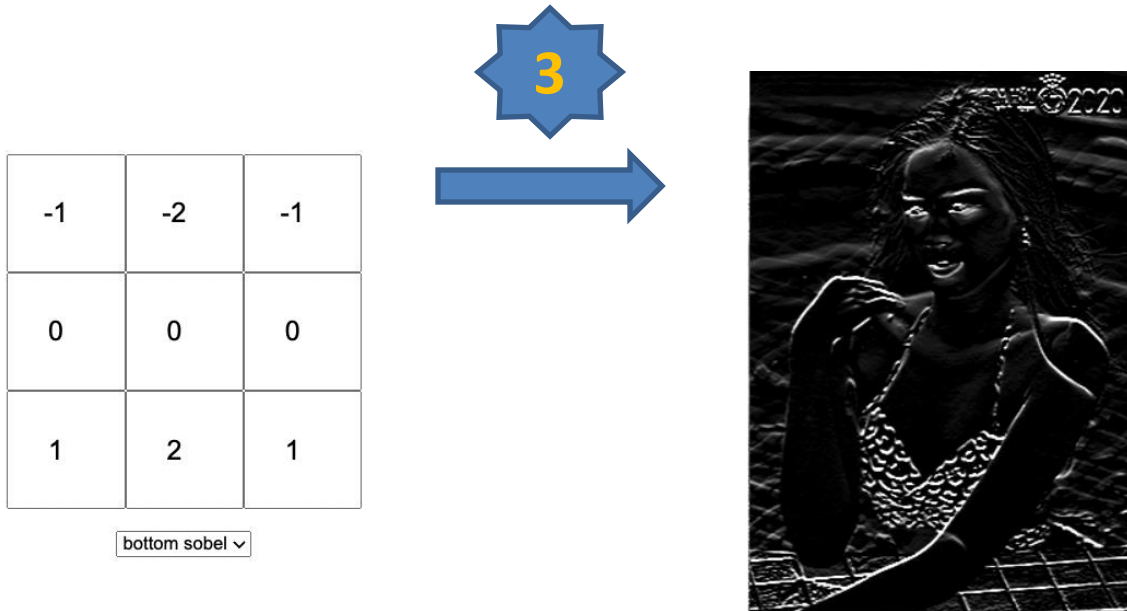
Thực hành 2.10

- Sinh viên đọc và hiển thị ảnh images/Thuchanh2_10.png ở chế độ ảnh xám.
- Áp dụng 4 bộ lọc sau trên ảnh. Lưu và hiển thị kết quả sau khi lọc.



Thực hành 2.10

- Áp dụng các bộ lọc sau trên ảnh. Lưu và hiển thị kết quả sau khi lọc.





QUESTIONS

Q & A

ANSWERS